RAG 벡터 검색 학습 노트

관련 파일: `dev2/nodes/vector_search.py` 작성일: 2026-06-05

한 줄 요약

이슈 텍스트를 숫자 벡터로 변환해 pgVector에서 의미적으로 유사한 과거 이슈를 찾고, 그 결과를 LLM 판단 노드의 컨텍스트로 주입하는 검색 강화 생성(RAG) 파이프라인.

왜 / 언제 쓰나

왜 필요한가:

- LLM은 사내 과거 PR 이력을 모른다. RAG 없이는 "SQL 인젝션 가능성 있음" 수준의 일반 경고만 가능.
- 과거 유사 사례를 주입하면 "3달 전 PR-002와 동일한 패턴, 당시 파라미터 바인딩으로 해결"이라는 맞춤 피드백 생성 가능.

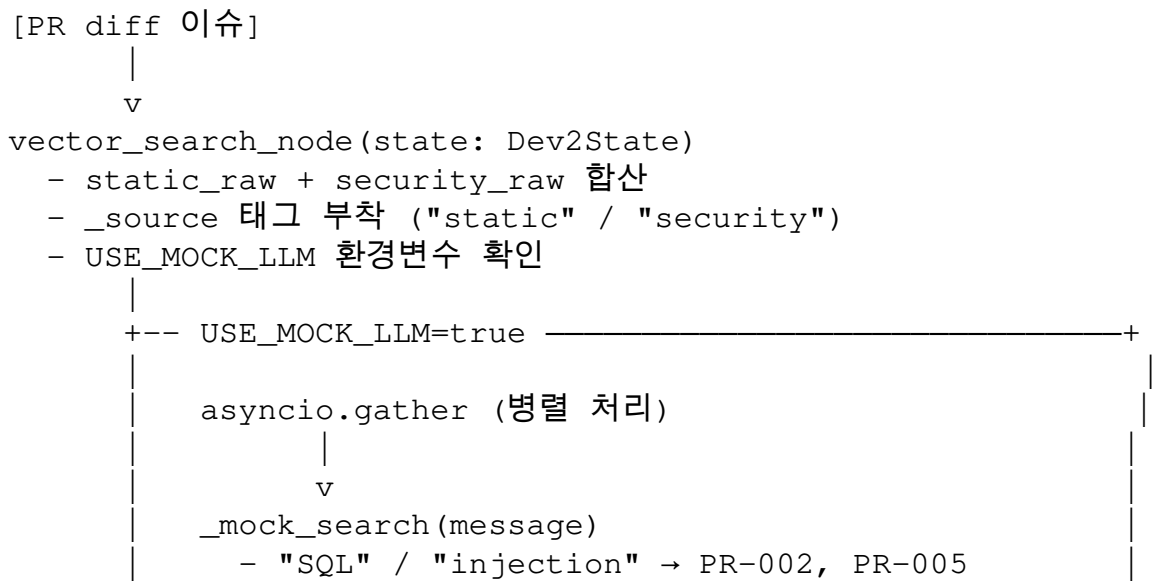
언제 실행되나:

- Dev1 Orchestrator가 PR diff를 받으면 Dev2가 정적/보안 분석을 수행.
- 분석 결과 이슈가 `vector_search_node`를 거쳐 `similar_issues` 필드가 추가된 enriched 형태로 `llm_judge`에 전달됨.

실제 **RAG** 동작 조건:

- `USE_MOCK_LLM=false` 설정
- `POSTGRES_URL` 환경변수 설정
- `chunk_embeddings` 테이블에 과거 이슈 seed 데이터 존재

코드 흐름 (단계별)



```
|     - "n+1" / "query"     → PR-003                  |
|     - "complexity"        → PR-001                  |
|     - 기타                → []                      |
|                                                     |
+-- USE_MOCK_LLM=false ──────────────────────────────+
      |
      v
_get_pool() → asyncpg 연결 풀 (POSTGRES_URL 필요)
      |
      v
asyncio.gather (병렬 처리)
      |
      v
_embed_text(message, use_mock=False)
  └── OpenAI text-embedding-3-small API 호출
      → 768차원 float 리스트
      (실패 시 _mock_embedding으로 fallback)
      |
      v
_vec_to_pg_str(vec)
  └── [0.1,0.2,...] 형식 문자열 변환
      (asyncpg가 list 직접 수용 불가)
      |
      v
_search_pgvector(query_vec, pool)
  └── SQL: 1-(embedding <=> $1::vector) >= 0.7
      ORDER BY 거리 ASC, LIMIT 3
      → [{pr_id, similarity, message}, ...]
      |
      v
_enrich_issue 결과: {**issue, "similar_issues": [...]}
      |
      v
{"enriched": [...]}  →  llm_judge 노드로 전달
```

핵심 SQL 구조

```
SELECT
    COALESCE(metadata->>'pr_id', chunk_id::text) AS pr_id,
    content,
    1 - (embedding <=> $1::vector)               AS similarity
FROM chunk_embeddings
WHERE 1 - (embedding <=> $1::vector) >= $2   -- 기본값: 0.7
ORDER BY embedding <=> $1::vector            -- 거리 오름차순 = 유사도 내림
LIMIT $3                                     -- 기본값: 3
```

- `<=>` : pgVector 코사인 거리 연산자
- `1 -` 거리 : 코사인 유사도 (1에 가까울수록 더 유사)
- `$1, $2, $3` : 파라미터 바인딩 (f-string SQL 조합 금지 — CLAUDE.md §6)

Mock 임베딩 동작 원리

```
def _mock_embedding(text: str) -> list[float]:
    digest = hashlib.md5(text.encode()).hexdigest()          # 32자리 16진
    seed_floats = [int(digest[i:i+2], 16) / 255.0            # 0~1 범위 16
                   for i in range(0, 32, 2)]
    vec = [seed_floats[i % len(seed_floats)]                 # 768차원으로
           for i in range(EMBEDDING_DIM)]
    norm = sum(x**2 for x in vec) ** 0.5 or 1.0
    return [x / norm for x in vec]                           # L2 정규화
```

- 동일 텍스트 → 동일 벡터 보장 (결정적, deterministic)
- 실제 API 없이 테스트 재현성 확보
- L2 정규화로 코사인 유사도 계산에 적합한 형태 유지

현재 프로젝트에서의 역할

구성 요소	역할
`vector_search_node`	LangGraph 노드 — Dev2 파이프라인의 enrichment 단계
`similar_issues` 필드	llm_judge 노드의 RAG 컨텍스트로 활용
`chunk_embeddings` 테이블	과거 이슈 벡터 저장소 (seed 데이터 필요)
Mock 모드	DB 없이 개발/테스트 가능 — Week 2~3 병렬 개발 지원
실제 모드	Week 5 통합 시 `USE_MOCK_LLM=false` + DB seed 데이터로 전환

환경변수 체크리스트 (실제 RAG 활성화):

```
USE_MOCK_LLM=false
OPENAI_API_KEY=sk-xxxx
POSTGRES_URL=postgresql://user:pass@host:5432/db
PGVECTOR_DIMENSION=768
VECTOR_SIMILARITY_THRESHOLD=0.7   # 기본값
VECTOR_TOP_K=3                    # 기본값
```

seed 데이터 필수 항목 (CLAUDE.md §8):

- N+1 쿼리 방지 가이드
- SQL 인젝션 방지 패턴
- 서비스 아키텍처 개요
- 신입 1주차 온보딩 가이드

오류 처리 패턴 (CLAUDE.md §12)

상황	처리 방법
`POSTGRES_URL` 미설정	`_get_pool()` → None 반환 → Mock 검색으로 fallback
asyncpg 미설치	ImportError 캐치 → None 반환 → Mock fallback

DB 연결 실패	`_get_pool()` → None 반환 → Mock fallback
pgVector 0건 검색	빈 리스트 반환, 예외 던지지 않음
OpenAI API 실패	`_mock_embedding`으로 fallback, 에러 로그 기록
개별 이슈 enrichment 실패	`similar_issues=[]`로 원본 이슈 유지, 나머지 계속 진행

복습 퀴즈

Q1. `USE_MOCK_LLM=true`일 때 "N+1 쿼리가 발생하고 있어요"라는 이슈 메시지가 들어오면 `_mock_search`는 무엇을 반환하나요? 이때 실제 DB 연결이나 OpenAI API 호출이 발생하나요?

정답 보기

메시지에 "n+1" 키워드가 포함되므로 `[{"pr_id": "PR-003", "similarity": 0.88, "message": "N+1 쿼리 — joinedload 사용 권장"}]`를 반환합니다. Mock 모드이므로 DB 연결도, OpenAI API 호출도 발생하지 않습니다.

Q2. `_vec_to_pg_str` 함수가 왜 필요한가요? Python의 `list[float]`를 바로 SQL 파라미터로 넘기면 안 되나요?

정답 보기

asyncpg는 Python `list`를 pgVector 파라미터로 직접 수용하지 않습니다.
`'[0.1,0.2,...]'` 형식의 문자열 + `::vector` 캐스팅을 조합해야 pgVector가 올바르게 인식합니다. `_vec_to_pg_str`이 이 변환을 담당합니다.

Q3. `SIMILARITY_THRESHOLD=0.7`의 의미는 무엇인가요? 이 값을 0.9로 올리면 어떤 일이 생기고, 0.3으로 낮추면 어떤 일이 생기나요?

정답 보기

코사인 유사도 0.7 이상인 과거 이슈만 `similar_issues`에 포함시킨다는 의미입니다.

- `0.9`로 올리면: 매우 유사한 사례만 반환되어 정밀도는 높아지지만, 유사 사례를 찾지 못하는 경우(빈 리스트)가 많아집니다.
- `0.3`으로 낮추면: 관련성이 낮은 사례도 포함되어 LLM에 노이즈가 주입되고, 피드백 품질이 낮아질 수 있습니다.